IT23302 | DATA STRUCTURES

SMART STUDENT RECORD & SEARCH SYSTEM

Detailed Mini Project Report

Project Description

A menu-driven C console application for maintaining student details and demonstrating the use of a Binary Search Tree (BST), Queue, searching techniques, sorting methods, and input validation.

Project detail	Information
Course	IT23302 – Data Structures
Project title	Smart Student Record & Search System
Programming language	C
Interface	Text-based console
Compiler	GCC or another C compiler
Project team	Student names and roll numbers to be filled in
Submission date	To be filled in

Prepared from the supplied mini project proposal. Optional features are identified as extensions and should only be claimed as implemented if they are present in the final program.

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Aim and Objectives
- 5. Scope of the Project
- 6. Tools and Technologies
- 7. System Requirements
- 8. Data Structures and Algorithms
- 9. System Design and Workflow
- 10. Planned Modules
- 11. Input Validation and Error Handling
- 12. Algorithm Complexity
- 13. Testing Plan
- 14. Expected Results
- 15. Advantages and Limitations
- 16. Future Enhancements
- 17. Conclusion
- Appendix A: Sample Console Interaction
- Appendix B: Project Completion Checklist

1. Abstract

The Smart Student Record & Search System is a proposed C-based console application intended to make basic student record management more organised. It stores a student's register number, name, department, and marks, and provides operations for adding, finding, editing, removing, displaying, and arranging records. The project demonstrates how suitable data structures can make a small record-management program easier to structure and maintain.

A Binary Search Tree (BST) is used as the principal structure, with register number serving as the key. The BST supports insertion, lookup, deletion, and inorder traversal, which displays records in ascending register-number order. A Queue supports student verification or processing requests according to the First In, First Out (FIFO) principle. Searching and sorting techniques provide additional ways to retrieve and arrange records. File storage is described as an optional extension, not a guaranteed feature of the base proposal.

2. Introduction

Student information is often needed for routine academic tasks such as verifying a register number, viewing department details, checking marks, or preparing an ordered list. When records are handled manually or stored without a clear structure, locating and comparing information can take unnecessary time. A simple computer-based system can organise these operations in one menu-driven interface.

This project is designed as a learning-oriented application rather than a full institutional database. Its main purpose is to demonstrate the practical application of data structures in C. The BST organises records by a unique register number, while the queue represents a separate workflow: students waiting for verification or processing. Search, sort, and validation functions complement these structures.

3. Problem Statement

The project addresses the difficulty of maintaining and retrieving student details in an organised way. The intended program should allow users to add records, locate a student, update or remove information, view records in an ordered form, and process waiting requests in arrival order.

The design should also reduce common data-entry errors. Duplicate register numbers can make it unclear which record is intended, and marks outside the permitted range should be rejected. The exact mark range should be selected and documented by the implementation team. Persistent storage may be added if the project scope and final implementation allow it.

4. Aim and Objectives

4.1 Aim

To design a menu-based C application for student record management that demonstrates the use of a Binary Search Tree, Queue, searching, sorting, and input validation.

4.2 Objectives

- Create a menu-based console interface for maintaining basic student details.
- Represent each student using a C structure containing register number, name, department, and marks.
- Use a BST keyed by register number for insertion, searching, deletion, and ordered traversal.
- Use a queue to maintain verification or processing requests in FIFO order.
- Provide search options for register number and, if implemented, student name.
- Sort records by marks, name, or another selected field using an appropriate sorting method.
- Validate input and reject duplicate register numbers and marks outside the chosen range.

- Evaluate the efficiency and limitations of the selected data structures.
- Consider file-based save/load as an optional extension.

5. Scope of the Project

5.1 Included in the base design

- Adding a student record with register number, name, department, and marks.
- Finding a record by register number using BST search.
- Removing a record from the BST and displaying records in ascending register-number order.
- Maintaining a processing queue and handling its front entry first.
- Searching and sorting features appropriate to the final implementation.
- Input checks for duplicates and invalid marks.

5.2 Optional or implementation-dependent

- Searching by student name using linear search.
- Editing an existing student's details.
- Saving records to a file and loading them at the next program start.
- Additional reports, filters, or sorting choices.

This report describes the planned design from the supplied proposal. A feature should be described as completed only after it has been implemented and tested in the actual C program.

6. Tools and Technologies

Tool / technology	Purpose
C programming language	Implements records, functions, BST nodes, queue operations, searching, sorting, and validation.
GCC or another C compiler	Compiles the source code into an executable program.
Text-based console	Displays the menu, prompts the user, and shows results or error messages.
Main memory	Stores BST nodes and queue entries while the program is running.
File operations (optional)	Can preserve records between sessions if save/load is implemented.

7. System Requirements

7.1 Hardware

- A desktop or laptop capable of running a C compiler.
- Sufficient memory and storage for the compiler, source files, and executable.

7.2 Software

- A C compiler such as GCC.
- A code editor or IDE, such as Visual Studio Code, configured to compile and run C programs.
- A terminal or console for interacting with the application.

These are general requirements for a small console program. The proposal does not specify a particular operating system or minimum hardware specification.

8. Data Structures and Algorithms

8.1 C Structure for Student Records

A C structure groups related fields into one logical record. The planned fields are register number, student name, department, and marks. Appropriate field sizes and data types should be selected by the implementation team, with input routines designed to avoid buffer overflow and accidental truncation.

8.2 Binary Search Tree (BST)

The BST is the primary record organisation. Each node represents a student, and the register number is used as the key. For a node with key K, smaller register numbers are placed in the left subtree and larger register numbers in the right subtree. Duplicate keys should be rejected or handled by a clearly defined rule; the proposal recommends checking for duplicates.

- **Insertion:** Compare the new register number with each visited node and continue left or right until an empty position is found.
- **Search:** Compare the requested register number with the current node. If it matches, return the record; otherwise continue into the appropriate subtree.
- **Deletion:** Handle the three standard cases: a leaf node, a node with one child, or a node with two children. For two children, replace the node with its inorder successor or predecessor and then remove that replacement node from its original position.
- **Inorder traversal:** Visit the left subtree, current node, and right subtree. Because the BST is keyed by register number, this produces ascending register-number order.

8.3 Queue

The queue stores students or requests awaiting verification or processing. It follows FIFO: the first request inserted is the first request removed. Enqueue adds a request at the rear; dequeue removes the request at the front; a display operation can show the pending entries. The implementation must handle an empty queue safely. If a fixed-size array is used, it must also handle a full queue; a linked queue can grow dynamically subject to available memory.

8.4 Searching

BST search is appropriate for register-number lookup. If name search is included, a linear scan through records is a straightforward option. Name matching rules—such as case sensitivity and handling of spaces—should be defined consistently.

8.5 Sorting

A sorting method such as bubble sort can arrange a temporary list of records by marks, name, or another selected field. Sorting by marks may be ascending or descending, but the program should state which order is used. Since the BST already provides register-number order through inorder traversal, a separate sorting routine is mainly useful for other fields or for demonstrating sorting algorithms.

8.6 File Operations (Optional)

File handling can be added to write records to a file and read them back at program startup. If implemented, the program should check whether a file opened successfully, detect read/write failures, and rebuild the BST or another in-memory representation from valid saved records. The proposal treats this as an extension.

9. System Design and Workflow

9.1 High-level architecture

Component	Responsibility
Menu and input layer	Shows choices, reads user input, and routes the request to a module.
Student record module	Creates, edits, validates, and displays student information.
BST module	Stores records by register number and supports insert, search, delete, and inorder traversal.
Queue module	Adds pending requests, processes the next request, and displays waiting entries.
Search/sort module	Provides name search if included and arranges records by the selected field.
Optional file module	Saves and restores records when persistent storage is implemented.

9.2 Typical workflow

- Start the program and initialise the BST root and queue state.
- Display the main menu and read the user's selected operation.
- For add, collect the student fields, validate them, check the register number, and insert the record if valid.
- For find or delete, request a register number and use BST operations to locate or remove the record.
- For view-all, perform inorder traversal to display records in ascending register-number order.
- For queue actions, enqueue a request, process the front request, or display pending requests.
- For sort, copy the records into a suitable array or list and apply the selected sorting algorithm.
- Display a success or error message and return to the menu until the user chooses Exit.
- If file storage is implemented, load saved records at startup and save changes at the appropriate time.

10. Planned Modules

Module	Purpose	Key consideration
Add record	Create a new student entry.	Reject duplicate register numbers and invalid data.
View all records	Display stored student entries.	Use inorder traversal for register-number order.
Find student	Retrieve a record.	Use BST search for register number; name search is optional.
Edit details	Modify an existing student's fields.	If register number changes, preserve BST ordering by removing and reinserting the node.
Remove record	Delete a student entry.	Handle leaf, one-child, and two-child cases.
Sort list	Arrange records by a chosen field.	Specify ascending/descending order and comparison rules.
Add to queue	Place a student/request in the waiting line.	Insert at rear.
Process next	Handle the next waiting entry.	Remove from front; report if the queue is empty.
View queue	Show pending entries.	Display in FIFO order.
Validate input	Check fields before modifying data.	Handle invalid numeric input and string lengths.
Save/load (optional)	Preserve records between runs.	Check file errors and invalid saved data.

11. Input Validation and Error Handling

Validation improves reliability by ensuring that only acceptable data enters the system. The exact rules must be reflected in both code and the final report.

- Register number: require a valid value and reject duplicates.
- Student name and department: reject empty values if required and limit input to the allocated field size.
- Marks: accept only numeric values within the range chosen for the project; a common academic range may be used only if confirmed by the project specification.
- Menu choice: detect non-numeric or out-of-range input and prompt again rather than performing an unintended operation.
- BST operations: handle an empty tree and report when a requested register number is not found.
- Queue operations: detect an empty queue before dequeueing; if using a fixed-size queue, detect overflow before enqueueing.
- Memory allocation: check whether dynamic allocation succeeds before accessing a new node.
- File operations (if included): check file-open and read/write results and avoid treating failed loads as valid empty data.

12. Algorithm Complexity

The following estimates describe typical implementations, not measured performance. Let n be the number of student records and h be the height of the BST.

Operation	Typical / expected cost	Worst case or note
BST search	O(h)	O(n) if the tree becomes skewed; O(log n) for a balanced tree.
BST insertion	O(h)	O(n) for a skewed tree; duplicates require a defined policy.
BST deletion	O(h)	O(n) for a skewed tree; deletion logic is more involved.
Inorder traversal	O(n)	Visits every node once.
Linear name search	O(n)	May inspect every record.
Bubble sort	$O(n^2)$	Common simple implementation; optimised version can stop early if already sorted.
Queue enqueue/dequeue	O(1)	Assuming a linked queue or a correctly maintained front/rear array queue.

A basic BST does not guarantee logarithmic performance. If the tree becomes highly unbalanced, operations can approach linear time. A self-balancing tree could improve worst-case performance, but it is outside the stated base design.

13. Testing Plan

Testing should verify both normal use and boundary conditions. The following is a suggested plan; results should be filled in after executing the actual program.

Test ID	Test input / action	Expected result
T1	Add a valid student record.	Record is inserted and can be displayed or found.
T2	Add another record with the same register number.	Duplicate is rejected or handled according to the documented rule.
T3	Search for an existing register number.	Correct student details are shown.
T4	Search for a register number not present.	Clear not-found message; program continues safely.
T5	Display all records after adding several entries out of order.	Inorder output is ascending by register number.
T6	Delete a leaf BST node.	Target is removed; remaining records stay searchable.
T7	Delete a node with one child and then one with two children.	Tree remains valid and traversal is ordered.
T8	Dequeue from an empty queue.	Program reports the queue is empty and does not crash.
T9	Enqueue A, then B; process next.	A is processed before B (FIFO).
T10	Enter invalid menu text or marks outside the chosen range.	Input is rejected and the program recovers.
T11	Sort records by marks.	Records appear in the documented marks order.
T12	Restart and reload records (only if file feature exists).	Previously saved valid records are restored.

Record actual observations, pass/fail status, and any corrections in the final submission. Do not report these expected results as observed test results until the program has been run.

14. Expected Results

The completed program is expected to offer a clear text-based menu for common student record operations. The BST should organise records by register number, support ordered traversal, and provide the basis for efficient lookup when the tree is reasonably balanced. The queue should process waiting requests in FIFO order. Search and sort options should make it possible to locate or arrange information using the supported fields.

Input validation is expected to reduce duplicate and invalid entries. If file storage is added, records can be retained between sessions. These are expected outcomes from the proposal and should be confirmed through implementation and testing.

15. Advantages and Limitations

15.1 Advantages

- Demonstrates practical use of a BST and queue in a relatable record-management task.
- Separates record organisation from processing-request order.
- BST inorder traversal naturally produces register-number order.
- Modular functions can make the code easier to understand, test, and maintain.
- Input validation can prevent common data-entry mistakes.
- Optional file handling can extend the program beyond a single session.

15.2 Limitations

- A plain BST may become skewed, making operations slower in the worst case.
- A console interface is less convenient than a graphical or web interface for some users.
- Without file storage, data held only in memory is lost when the program ends.
- Bubble sort is simple but inefficient for large datasets.
- The base proposal does not specify authentication, user roles, backups, or privacy controls.
- The system is a small educational project, not a replacement for a secure institutional student-information platform.

16. Future Enhancements

- Add robust file-based persistence with clear error handling.
- Use a self-balancing search tree if predictable search performance is required.
- Add filters and reports, such as students above a chosen mark threshold.
- Support more flexible name search and clearly documented matching rules.
- Add a graphical user interface if the project scope permits.
- Introduce authentication and access controls only if the system is expanded for real users.
- Improve testing with larger datasets and documented performance measurements.

17. Conclusion

The Smart Student Record & Search System is a practical mini-project for demonstrating core Data Structures concepts through a student record-management scenario. The BST organises records by register number and supports insertion, search, deletion, and ordered traversal. The queue models a waiting line using FIFO behaviour, while searching, sorting, and validation provide useful supporting operations.

The proposal establishes a clear foundation for a modular C console application. The quality of the final project will depend on correct implementation of BST deletion, safe queue handling, consistent input validation, and systematic testing. File storage and other enhancements should be reported as completed only if they are implemented and verified.

Appendix A: Sample Console Interaction

The following is illustrative output showing the intended style of interaction. It is not evidence that the program has been executed.

```
SMART STUDENT RECORD & SEARCH SYSTEM
1. Add student record
2. View all records
3. Find student
4. Edit student details
5. Remove student record
6. Sort records
7. Add student to processing queue
8. Process next student
9. View pending queue
0. Exit

Enter choice: 1
Register number: 2401
Student name: Example Student
Department: IT
Marks: 86
Record added successfully.

Enter choice: 3
Enter register number to search: 2401
Register No: 2401
Name: Example Student
Department: IT
Marks: 86
```

Appendix B: Project Completion Checklist

- Fill in project team names, roll numbers, and submission date.
- Confirm the exact allowed marks range and field sizes.
- Compile the C source with the chosen compiler and resolve all warnings/errors.
- Test BST insertion, search, deletion, and inorder traversal.
- Test queue enqueue, dequeue, FIFO order, empty queue, and overflow handling if applicable.
- Test sorting and searching options actually present in the code.

- Run all relevant test cases and record actual pass/fail outcomes.
- Mark optional features such as file save/load as implemented only after successful testing.
- Update this report to match the final code and actual observed results.